claude-windows / dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3\subagents\agent-af0ea96840b5dd1f6.jsonl`
- SHA-256: `e950262fd760450681855663dfb0cfebc196d4eda18447e4a135382469a5fa1c`
- Source modified: `2026-06-21T09:07:53+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `subagents`
- session_id: `dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3`

Transcript

1. user (2026-06-21T09:04:40.502Z)

Review milestone M4 of COMPUTE_DECOUPLED_SERVING in the repo at C:/Users/avidu/Projects/badminton-highlight-indexer. Run `git diff` (working tree vs HEAD) to see the change.

M4 = "DeviceContext + WASB CPU-fallback; unified healthcheck/device wiring; SAFE (cuda default unchanged)". It adds `backend/pipeline/detectors/device.py::DeviceContext` and wires it into `backend/pipeline/detectors/native_wasb_runner.py` so the runner supports a new `device="auto"` value (cuda-if-available-else-cpu) ? the CPU fallback for a no-GPU box ? while keeping explicit `device="cuda"` strict. `indexing.wasb.device` doc updated. New tests: `tests/test_device_context.py` + additions to `tests/test_native_wasb_runner.py`.

Verify ONLY these, grounded in code with file:line:
1. SAFETY / "cuda default unchanged": Does an EXPLICIT `device="cuda"` on a GPU-less box still HARD-FAIL the healthcheck (not silently downgrade to cpu)? Is the default still "cuda"? Confirm no behaviour change for any pre-M4 config (cuda/cpu/mps). Is there ANY path where an explicit cuda silently runs on cpu?
2. "auto" NEVER reaches wasb_infer.py: `wasb_infer.py` would mis-handle a literal "auto" device. Confirm run_predict always resolves auto?cuda/cpu BEFORE building the `--device` argv, on every path (healthcheck-first AND run_predict-without-healthcheck).
3. PROBE CACHING & CORRECTNESS: healthcheck probes cuda and caches it (`self._cuda_available`); run_predict reuses the cache (no 2nd subprocess) but probes lazily if healthcheck was skipped. Is the cache coherent? Could a stale/None cache cause a wrong device? Does `_probe_cuda` correctly treat python-missing / timeout / import-error / rc!=0 as "no cuda" (so auto?cpu, never a crash)?
4. EDGE CASES: device="auto" but the torch import itself fails (rc!=0) in healthcheck ? what happens? device="mps" (passthrough)? An unknown device string (e.g. "gpu")? Does the warning fire only on a real fallback?
5. TEST GAPS / OS: are the safety contract (explicit-cuda-strict), the auto?cpu fallback, the auto?cuda, the no-probe-for-explicit, and the cache-reuse all pinned by a test that would FAIL on regression? Any OS-dependent assertion (this runs cross-OS in CI)? Is DeviceContext's frozen/immutability and each property covered?

Report concrete findings with severity (blocker/major/minor/nit) + a fix. A clean bill is valid. Do NOT edit ? review only.

2. assistant (2026-06-21T09:04:42.982Z)

I'll review M4 of COMPUTE_DECOUPLED_SERVING. Let me start by pulling the latest and examining the diff.

3. user (2026-06-21T09:04:44.779Z)

```
 backend/config/models.py                          |  4 +-
 backend/pipeline/detectors/native_wasb_runner.py | 55 ++++++++++++++---
 tests/test_native_wasb_runner.py                  | 71 +++++++++++++++++++++
```

```
 3 files changed, 122 insertions(+), 8 deletions(-)
```

4. user (2026-06-21T09:04:45.269Z)

```
feat/serving-m3-output-base
aa987cd docs(serving): mark §7 tracker M3 ? + changelog (PR #282)
41b7094 feat(serving): M3 ? configurable output/scratch base (de-hardcode output/, default-OFF)
41d45fd fix(server): drain async jobs under `python -m backend.main` (double-module hang) (#281)
```

5. assistant (2026-06-21T09:04:47.989Z)

I'm on branch `feat/serving-m3-output-base`, but the diff stat shows uncommitted working-tree changes ? that's M4. Let me read the full diff and the relevant files.

6. user (2026-06-21T09:04:48.675Z)

```
diff --git a/backend/config/models.py b/backend/config/models.py
index a31c9da..92ad880 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -69,7 +69,9 @@ class WasbIndexConfig(BaseModel):
     velocity_threshold: float = 0.02
     # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner. ---
     # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
-    device: str = "cuda"        # torch device for the native runner: cuda | mps | cpu
+    # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4 CPU
fallback
+    # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-CPU
downgrade).
+    device: str = "cuda"        # auto | cuda | mps | cpu
     python_bin: str = "python"  # the `wasb` conda env python (invoked by path, no `conda
activate`)
     # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
     # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates, regenerable
diff --git a/backend/pipeline/detectors/native_wasb_runner.py
b/backend/pipeline/detectors/native_wasb_runner.py
index f8fc1ba..30494a0 100644
--- a/backend/pipeline/detectors/native_wasb_runner.py
+++ b/backend/pipeline/detectors/native_wasb_runner.py
@@ -10,8 +10,9 @@ see the acceptance gate below) ? but WITHOUT the WSL transport:
   * NO ``WslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
     ``wasb`` env's python is invoked **by path**), NO ``/mnt/c`` path translation
     (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
-  * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps`` or ``cpu``; threaded
-    into ``wasb_infer.py --device`` (and so into the Hydra ``runner.device`` override).
+  * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps``, ``cpu``, or ``auto``
+    (M4: cuda-if-available-else-cpu ? the CPU fallback for a no-GPU box; explicit ``cuda``
stays
+    strict); threaded into ``wasb_infer.py --device`` (and so the Hydra ``runner.device``
override).
   * weights / repo / python / scratch come from the **environment** first
     (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
     ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.
@@ -40,6 +41,7 @@ from dataclasses import dataclass
 from typing import Any, Dict, List, Optional, Tuple

 from backend.pipeline.detectors.base import DetectorRunner
+from backend.pipeline.detectors.device import AUTO, DeviceContext
 # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

@@ -62,7 +64,7 @@ class NativeWasbConfig:
     python_bin: str = "python"             # the `wasb` conda env python (invoked by path, no
```

```
activate)
     weights_path: str = ""                # REQUIRED (env WASB_WEIGHTS or
indexing.wasb.weights_path)
     sport: str = "badminton"
-    device: str = "cuda"                  # cuda | mps | cpu
+    device: str = "cuda"                  # auto | cuda | mps | cpu (auto = cuda-if-available-
else-cpu)
     scratch_dir: str = ""                 # frame-extraction scratch (env); "" = next to the
output dir
     timeout_sec: int = 1800              # 30 min; a hung GPU call is killed (0 = no timeout)
     keep_frames: bool = False             # retain the decoded frame cache after success
(debug only)
@@ -96,6 +98,9 @@ class NativeWasbRunner(DetectorRunner):

     def __init__(self, cfg: NativeWasbConfig):
         self.cfg = cfg
+        # Cached CUDA-availability probe (used to resolve device="auto"). Set by healthcheck;
+        # lazily probed by run_predict if healthcheck was skipped. None = not yet probed.
+        self._cuda_available: Optional[bool] = None

     # --- low-level native exec (the single place tests patch) -------------------------
     def _run(self, argv: List[str], cwd: Optional[str] = None) -> subprocess.CompletedProcess:
@@ -103,6 +108,30 @@ class NativeWasbRunner(DetectorRunner):
         return subprocess.run(argv, cwd=cwd, capture_output=True, text=True,
                               timeout=(self.cfg.timeout_sec or None))

+    # --- device resolution (M4 DeviceContext: device="auto" ? cuda-if-available-else-cpu)
------
+    def _probe_cuda(self) -> bool:
+        """Probe ``torch.cuda.is_available()`` in the wasb env's python (a subprocess). Any
failure
+        (missing python / timeout / import error) is treated as 'no CUDA'."""
+        code = "import torch; print('cuda', torch.cuda.is_available())"
+        try:
+            res = self._run([self.cfg.python_bin, "-c", code])
+        except (FileNotFoundError, subprocess.TimeoutExpired):
+            return False
+        return res.returncode == 0 and "cuda True" in (res.stdout or "")
+
+    def _cuda_is_available(self) -> bool:
+        """CUDA availability, cached across healthcheck + run_predict so we probe at most
once."""
+        if self._cuda_available is None:
+            self._cuda_available = self._probe_cuda()
+        return self._cuda_available
+
+    def _effective_device(self) -> str:
+        """The torch device to actually pass to ``wasb_infer.py``. Only ``auto`` needs the CUDA
+        probe; an explicit cuda/mps/cpu resolves to itself (so ``device='cuda'`` is
unchanged)."""
+        if self.cfg.device == AUTO:
+            return DeviceContext(AUTO, self._cuda_is_available()).effective
+        return self.cfg.device
+
     def _repo_src(self) -> str:
         return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")

@@ -143,8 +172,16 @@ class NativeWasbRunner(DetectorRunner):
         out = (res.stdout or "").strip()
         if res.returncode != 0:
             return False, f"`{self.cfg.python_bin}` torch import failed: {(res.stderr or
out).strip()}"
-        if self.cfg.device == "cuda" and "cuda True" not in out:
+        # M4: resolve the device. Cache the probe so run_predict reuses it (no second
subprocess).
```

```
+        self._cuda_available = "cuda True" in out
+        ctx = DeviceContext(self.cfg.device, self._cuda_available)
+        if ctx.cuda_required_but_missing:
+            # Unchanged pre-M4 behaviour for an EXPLICIT cuda request: hard-fail, never a
silent
+            # slow-CPU downgrade. Set indexing.wasb.device=auto for a CPU fallback.
             return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
+        if ctx.fell_back:
+            logger.warning("device=auto: no CUDA GPU detected (%s) ? FALLING BACK TO CPU; WASB
"
+                           "inference will be slow. Set indexing.wasb.device=cuda to require a
GPU.", out)
         return True, out

     def run_predict(self, video_win_path: str, output_win_dir: str,
@@ -179,13 +216,17 @@ class NativeWasbRunner(DetectorRunner):
             scratch = self.cfg.scratch_dir or output_dir
             frames_dir = os.path.join(scratch, f"{key}_frames")
             argv += ["--frames_out_dir", frames_dir]
+        # M4: resolve device="auto" to the effective device (cuda-if-available-else-cpu); an
+        # explicit cuda/mps/cpu passes through unchanged. Never sends "auto" to wasb_infer.py.
+        device = self._effective_device()
         argv += ["--weights", weights,
                  "--sport", self.cfg.sport,
-                 "--device", self.cfg.device,
+                 "--device", device,
                  "--out", out_csv]

-        logger.info("Running native WASB inference (device=%s) on %s ...",
-                    self.cfg.device, os.path.basename(norm))
+        logger.info("Running native WASB inference (device=%s%s) on %s ...",
+                    device, " [auto?fallback]" if self.cfg.device == AUTO and device != "cuda"
else "",
+                    os.path.basename(norm))
         try:
             res = self._run(argv, cwd=self._repo_src())
         except FileNotFoundError:
diff --git a/tests/test_native_wasb_runner.py b/tests/test_native_wasb_runner.py
index 22a3abe..2f71f7a 100644
--- a/tests/test_native_wasb_runner.py
+++ b/tests/test_native_wasb_runner.py
@@ -6,6 +6,7 @@ the GPU-box byte-parity gate then relies on: it IS a DetectorRunner, it is NOT a
 activate`, no `/mnt` translation) that runs the SAME wasb_infer.py with the SAME output CSV
 name as the WSL runner, threads the device through, and cleans the frame cache on success.
 """
+import logging
 import os
 import types
 from unittest.mock import patch
@@ -134,6 +135,76 @@ def test_healthcheck_python_not_found(tmp_path):
     assert not ok and "python" in msg.lower()


+# --- M4: device="auto" CPU fallback (DeviceContext) ------------------------------------
+def test_healthcheck_auto_uses_cuda_when_available(tmp_path):
+    r = NativeWasbRunner(_ok_cfg(tmp_path, device="auto"))
+    with patch.object(r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda True")):
+        ok, _msg = r.healthcheck()
+    assert ok
+    assert r._cuda_available is True
+    assert r._effective_device() == "cuda"   # auto resolves to the GPU when present
+
+
+def test_healthcheck_auto_falls_back_to_cpu_with_warning(tmp_path, caplog):
+    """device=auto on a GPU-less box does NOT hard-fail (unlike device=cuda) ? it resolves to
```

```
cpu
+    and warns loudly. This is the M4 portability win for a no-GPU Linux / cpu-serving box."""
+    r = NativeWasbRunner(_ok_cfg(tmp_path, device="auto"))
+    with patch.object(r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda False")), \
+         caplog.at_level(logging.WARNING,
logger="backend.pipeline.detectors.native_wasb_runner"):
+        ok, _msg = r.healthcheck()
+    assert ok                                  # auto does not require a GPU
+    assert r._effective_device() == "cpu"
+    assert any("FALLING BACK TO CPU" in rec.message for rec in caplog.records)
+
+
+def test_healthcheck_caches_probe_for_run_predict(tmp_path):
+    """healthcheck's CUDA probe is cached so run_predict reuses it (no second subprocess)."""
+    r = NativeWasbRunner(_ok_cfg(tmp_path, device="auto"))
+    with patch.object(r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda False")) as
spy:
+        r.healthcheck()
+        assert spy.call_count == 1
+        assert r._effective_device() == "cpu"   # uses the cache, no extra probe
+        assert spy.call_count == 1
+
+
+def test_run_predict_auto_threads_cpu_when_no_gpu(tmp_path):
+    """run_predict with device=auto + no GPU: the lazy probe (mocked empty stdout = no cuda)
+    resolves to cpu, and 'auto' is NEVER sent to wasb_infer.py."""
+    _, _out, _rm, argv, _cwd, _key = _drive_success(
+        NativeWasbConfig(weights_path="/m/w.pth", device="auto"), tmp_path)
+    assert argv[argv.index("--device") + 1] == "cpu"
+    assert "auto" not in argv
+
+
+def test_run_predict_auto_threads_cuda_when_gpu(tmp_path):
+    """run_predict with device=auto + a GPU: the probe reports cuda True ? --device cuda."""
+    r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth", device="auto"))
+    key = "clip__abc123abc123"
+    (tmp_path / f"{key}_wasb.csv").write_text("Frame,Visibility,X,Y\n")
+
+    def _run_side(argv, cwd=None):
+        if "-c" in argv:                       # the CUDA probe
+            return _proc(0, stdout="cuda True")
+        return _proc(0)                        # the inference
+
+    with patch.object(r, "_copy_infer_script", return_value=True), \
+         patch.object(r, "_run", side_effect=_run_side) as spy, \
+         patch.object(r, "_rm_rf"):
+        out = r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
+    assert out is not None
+    infer_argv = next(c.args[0] for c in spy.call_args_list if "wasb_infer.py" in c.args[0])
+    assert infer_argv[infer_argv.index("--device") + 1] == "cuda"
+
+
+def test_run_predict_explicit_cuda_does_not_probe(tmp_path):
+    """An explicit device=cuda resolves to itself with NO probe subprocess ? unchanged
behaviour
+    (a single _run call: the inference)."""
+    r, _out, _rm, argv, _cwd, _key = _drive_success(
+        NativeWasbConfig(weights_path="/m/w.pth", device="cuda"), tmp_path)
+    assert argv[argv.index("--device") + 1] == "cuda"
+    assert r._cuda_available is None           # never probed for an explicit device
+
+
 # --- run_predict ------------------------------------------------------------
 def _drive_success(cfg, tmp_path, vid="abc123abc123", **patches):
     """Drive run_predict to success with the subprocess mocked; returns (runner, out, rm_spy,
```

```
argv, cwd)."""
```

7. user (2026-06-21T09:04:49.044Z)

```
On branch feat/serving-m3-output-base
Your branch is based on 'origin/feat/serving-m3-output-base', but the upstream is gone.
  (use "git branch --unset-upstream" to fixup)

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   backend/config/models.py
        modified:   backend/pipeline/detectors/native_wasb_runner.py
        modified:   tests/test_native_wasb_runner.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        artifacts/
        backend/pipeline/detectors/device.py
        docs/KHELSUTRA_PRODUCT_OVERVIEW.md
        tests/test_device_context.py

no changes added to commit (use "git add" and/or "git commit -a")
```

8. assistant (2026-06-21T09:04:51.706Z)

The diff doesn't show the new untracked files (`device.py` and `test_device_context.py`). Let me read those plus the full `native_wasb_runner.py` to ground my review in actual line numbers.

9. user (2026-06-21T09:04:52.190Z)

```
1       """DeviceContext ? resolve the effective torch device for a detector runner.
2
3       COMPUTE_DECOUPLED_SERVING M4 (realizes the ``DeviceContext`` designed in the archived
4       ``07-COMPUTE-ABSTRACTION``). A runner asks for a device by name; this turns the
*requested*
5       device + a probed CUDA availability into the *effective* device to actually run on, and
says
6       whether a CPU fallback happened.
7
8       The portability gap it closes: the native WASB runner hardcoded ``device='cuda'`` and
its
9       healthcheck hard-failed on a box with no GPU, so a CPU-only Linux box (cpu-serving / a
10      no-GPU truly-local install / CI) could not run at all.
11
12      **Safety contract (so "cuda default unchanged" holds):**
13        * ``"auto"``  ? ``cuda`` if a GPU is available, else ``cpu`` (the fallback). This is
opt-in.
14        * ``"cuda"``  ? honored verbatim; an *explicit* cuda request on a GPU-less box still
hard-fails
15          elsewhere (no silent slow-CPU downgrade of a config that asked for the GPU).
16        * ``"cpu"`` / ``"mps"`` ? honored verbatim.
17      """
18
19      from __future__ import annotations
20
21      from dataclasses import dataclass
22
23      AUTO = "auto"
24      CUDA = "cuda"
25      CPU = "cpu"
26
27
28      @dataclass(frozen=True)
```

```
29      class DeviceContext:
30          """The requested device + a probed CUDA availability ? the effective device."""
31
32          requested: str          # what config/env asked for: auto | cuda | mps | cpu
33          cuda_available: bool     # probed in the runner's own torch env (not the main
process)
34
35          @property
36          def effective(self) -> str:
37              """The device the runner should actually use. ``auto`` resolves to cuda-if-
available-else-cpu;
38              any explicit device is returned verbatim."""
39              if self.requested == AUTO:
40                  return CUDA if self.cuda_available else CPU
41              return self.requested
42
43          @property
44          def fell_back(self) -> bool:
45              """True when ``auto`` resolved to CPU because no GPU was found (worth a loud
warning)."""
46              return self.requested == AUTO and not self.cuda_available
47
48          @property
49          def cuda_required_but_missing(self) -> bool:
50              """True when an EXPLICIT ``cuda`` request can't be satisfied ? the caller should
hard-fail
51              rather than silently downgrade (preserving the pre-M4 strict behaviour for
``device='cuda'``)."""
52              return self.requested == CUDA and not self.cuda_available
53
```

10. user (2026-06-21T09:04:52.552Z)

```
1       """DeviceContext ? torch device resolution for detector runners
(COMPUTE_DECOUPLED_SERVING M4).
2
3       Pins the safety contract: ``auto`` falls back cuda?cpu (the new portability win), while
an
4       explicit ``cuda`` is honored verbatim and flagged as required-but-missing (the caller
hard-fails,
5       no silent slow-CPU downgrade ? "cuda default unchanged").
6       """
7       from backend.pipeline.detectors.device import DeviceContext
8
9
10      def test_auto_resolves_to_cuda_when_available():
11          ctx = DeviceContext("auto", cuda_available=True)
12          assert ctx.effective == "cuda"
13          assert ctx.fell_back is False
14          assert ctx.cuda_required_but_missing is False
15
16
17      def test_auto_falls_back_to_cpu_when_no_gpu():
18          ctx = DeviceContext("auto", cuda_available=False)
19          assert ctx.effective == "cpu"
20          assert ctx.fell_back is True
21          assert ctx.cuda_required_but_missing is False
22
23
24      def test_explicit_cuda_is_honored_verbatim():
25          assert DeviceContext("cuda", cuda_available=True).effective == "cuda"
26          miss = DeviceContext("cuda", cuda_available=False)
27          assert miss.effective == "cuda"                # honored ? NOT silently downgraded
to cpu
28          assert miss.cuda_required_but_missing is True    # ?so the caller hard-fails
```

```
29          assert miss.fell_back is False
30
31
32      def test_explicit_cpu_and_mps_pass_through():
33          assert DeviceContext("cpu", cuda_available=False).effective == "cpu"
34          assert DeviceContext("cpu", cuda_available=True).effective == "cpu"
35          assert DeviceContext("mps", cuda_available=False).effective == "mps"
36          for d in ("cpu", "mps"):
37              ctx = DeviceContext(d, cuda_available=False)
38              assert ctx.fell_back is False
39              assert ctx.cuda_required_but_missing is False
40
41
42      def test_is_frozen():
43          import dataclasses
44          import pytest
45          ctx = DeviceContext("auto", cuda_available=True)
46          with pytest.raises(dataclasses.FrozenInstanceError):
47              ctx.requested = "cpu"  # type: ignore[misc]
48
```

11. assistant (2026-06-21T09:04:55.728Z)

Now let me read the full `native_wasb_runner.py` to see the healthcheck and run_predict in context, especially the ordering of operations and how `healthcheck` is (or isn't) called before `run_predict`.

12. user (2026-06-21T09:04:56.724Z)

```
1       """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3       The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4       "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
5       It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
6       Linux GPU box (GCP / a SaaS worker): it runs the **same** verified ``wasb_infer.py``
core
7       that the WSL adapter runs, so the trajectory CSV is identical (to a sub-pixel float
tolerance;
8       see the acceptance gate below) ? but WITHOUT the WSL transport:
9
10        * NO ``WslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11          ``wasb`` env's python is invoked **by path**), NO ``/mnt/c`` path translation
12          (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
13        * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps``, ``cpu``, or
``auto``
14          (M4: cuda-if-available-else-cpu ? the CPU fallback for a no-GPU box; explicit
``cuda`` stays
15          strict); threaded into ``wasb_infer.py --device`` (and so the Hydra
``runner.device`` override).
16        * weights / repo / python / scratch come from the **environment** first
17          (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
18          ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.
19
20      It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``
21      name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
22      ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
23      the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
24      (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
``detector_impl="native"``.
25
26      **Acceptance gate (parity, NOT strict byte-equality):** the SAME clip through the WSL
runner
27      (Windows) and this runner (Linux, ``device=cuda``) yields the same trajectory CSV + the
same
```

```
    candidate windows **within a small float tolerance (~1e-3 px)**. cuDNN is non-
deterministic
    ACROSS GPUs, so a sub-pixel diff is expected on different hardware ? but it is byte-
exact
    run-to-run on a FIXED GPU. ? Validated 2026-06-20 on an RTX 2070 (via WSL): native
streaming
    vs the cached WSL-disk CSV matched **1194/1195** frames (the lone diff a ~5e-5 px edge
    artifact), and two native runs were byte-identical (deterministic). This is a hardware
check
    (a real GPU + the ``wasb`` env), so the offline, subprocess-mocked suite asserts the
contract,
    not the numbers.
    """
    import os
    import shutil
    import logging
    import subprocess
    from dataclasses import dataclass
    from typing import Any, Dict, List, Optional, Tuple

    from backend.pipeline.detectors.base import DetectorRunner
    from backend.pipeline.detectors.device import AUTO, DeviceContext
    # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

    logger = logging.getLogger(__name__)

    # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
    # detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
    _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")


    @dataclass
    class NativeWasbConfig:
        """Resolved settings for a Linux-native WASB run.

        Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
        over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
``device``
        defaults to ``cuda`` (the WSL parity path); ``stream_video`` defaults to True (the
perf win).
        """
        repo_dir: str = "~/models/WASB-SBDT"   # native clone of nttcom/WASB-SBDT
        python_bin: str = "python"             # the `wasb` conda env python (invoked by
path, no activate)
        weights_path: str = ""                 # REQUIRED (env WASB_WEIGHTS or
indexing.wasb.weights_path)
        sport: str = "badminton"
        device: str = "cuda"                   # auto | cuda | mps | cpu (auto = cuda-if-
available-else-cpu)
        scratch_dir: str = ""                  # frame-extraction scratch (env); "" = next
to the output dir
        timeout_sec: int = 1800               # 30 min; a hung GPU call is killed (0 = no
timeout)
        keep_frames: bool = False              # retain the decoded frame cache after
success (debug only)
        # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long
clip's
        # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a
real GPU
        # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default
here.
        stream_video: bool = True

```

```
76          @classmethod
77          def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "NativeWasbConfig":
78              w = (idx_cfg or {}).get("wasb", {}) or {}
79              env = os.environ.get
80              # stream_video is tri-state in config: None/absent => native default (stream);
an
81              # explicit True/False still wins (e.g. False for a container cv2 can't seek).
82              sv = w.get("stream_video")
83              return cls(
84                  repo_dir=env("WASB_REPO_DIR") or w.get("repo_dir", "~/models/WASB-SBDT"),
85                  python_bin=env("WASB_PYTHON") or w.get("python_bin", "python"),
86                  weights_path=env("WASB_WEIGHTS") or w.get("weights_path", "") or "",
87                  sport=w.get("sport", "badminton"),
88                  device=env("WASB_DEVICE") or w.get("device", "cuda"),
89                  scratch_dir=env("WASB_SCRATCH") or env("RALLY_SCRATCH") or env("TMPDIR") or
"",
90                  timeout_sec=int(w.get("timeout_sec", 1800)),
91                  keep_frames=bool(w.get("keep_frames", False)),
92                  stream_video=(True if sv is None else bool(sv)),
93              )
94
95
96      class NativeWasbRunner(DetectorRunner):
97          """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
docstring."""
98
99          def __init__(self, cfg: NativeWasbConfig):
100             self.cfg = cfg
101             # Cached CUDA-availability probe (used to resolve device="auto"). Set by
healthcheck;
102             # lazily probed by run_predict if healthcheck was skipped. None = not yet
probed.
103             self._cuda_available: Optional[bool] = None
104
105         # --- low-level native exec (the single place tests patch)
--------------------------
106         def _run(self, argv: List[str], cwd: Optional[str] = None) ->
subprocess.CompletedProcess:
107             logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
108             return subprocess.run(argv, cwd=cwd, capture_output=True, text=True,
109                                   timeout=(self.cfg.timeout_sec or None))
110
111         # --- device resolution (M4 DeviceContext: device="auto" ? cuda-if-available-else-
cpu) ------
112         def _probe_cuda(self) -> bool:
113             """Probe ``torch.cuda.is_available()`` in the wasb env's python (a subprocess).
Any failure
114             (missing python / timeout / import error) is treated as 'no CUDA'."""
115             code = "import torch; print('cuda', torch.cuda.is_available())"
116             try:
117                 res = self._run([self.cfg.python_bin, "-c", code])
118             except (FileNotFoundError, subprocess.TimeoutExpired):
119                 return False
120             return res.returncode == 0 and "cuda True" in (res.stdout or "")
121
122         def _cuda_is_available(self) -> bool:
123             """CUDA availability, cached across healthcheck + run_predict so we probe at
most once."""
124             if self._cuda_available is None:
125                 self._cuda_available = self._probe_cuda()
126             return self._cuda_available
127
128         def _effective_device(self) -> str:
129             """The torch device to actually pass to ``wasb_infer.py``. Only ``auto`` needs
the CUDA
```

```
130             probe; an explicit cuda/mps/cpu resolves to itself (so ``device='cuda'`` is
unchanged)."""
131             if self.cfg.device == AUTO:
132                 return DeviceContext(AUTO, self._cuda_is_available()).effective
133             return self.cfg.device
134
135         def _repo_src(self) -> str:
136             return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
137
138         def _copy_infer_script(self) -> bool:
139             """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
configs/."""
140             repo_src = self._repo_src()
141             try:
142                 os.makedirs(repo_src, exist_ok=True)
143                 shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
144                 return True
145             except OSError as e:
146                 logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
147                 return False
148
149         def _rm_rf(self, path: Optional[str]) -> None:
150             """Best-effort recursive delete of a native path (post-success cleanup; never
raises)."""
151             if path:
152                 shutil.rmtree(path, ignore_errors=True)
153
154         def healthcheck(self) -> Tuple[bool, str]:
155             """Verify weights + repo present and the `wasb` python imports torch (and sees a
GPU on cuda)."""
156             if not self.cfg.weights_path:
157                 return False, ("WASB weights path is empty ? set WASB_WEIGHTS (or
indexing.wasb.weights_path).")
158             weights = os.path.expanduser(self.cfg.weights_path)
159             if not os.path.exists(weights):
160                 return False, f"WASB weights not found at {weights}"
161             repo_src = self._repo_src()
162             if not os.path.isdir(repo_src):
163                 return False, (f"WASB-SBDT repo src/ not found at {repo_src} ? clone
nttcom/WASB-SBDT "
164                                f"(set WASB_REPO_DIR / indexing.wasb.repo_dir).")
165             code = "import torch; print('torch', torch.__version__, 'cuda',
torch.cuda.is_available())"
166             try:
167                 res = self._run([self.cfg.python_bin, "-c", code])
168             except FileNotFoundError:
169                 return False, f"python binary not found: {self.cfg.python_bin!r} (set
WASB_PYTHON)."
170             except subprocess.TimeoutExpired:
171                 return False, "native torch healthcheck timed out."
172             out = (res.stdout or "").strip()
173             if res.returncode != 0:
174                 return False, f"`{self.cfg.python_bin}` torch import failed: {(res.stderr or
out).strip()}"
175             # M4: resolve the device. Cache the probe so run_predict reuses it (no second
subprocess).
176             self._cuda_available = "cuda True" in out
177             ctx = DeviceContext(self.cfg.device, self._cuda_available)
178             if ctx.cuda_required_but_missing:
179                 # Unchanged pre-M4 behaviour for an EXPLICIT cuda request: hard-fail, never
a silent
180                 # slow-CPU downgrade. Set indexing.wasb.device=auto for a CPU fallback.
181                 return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
182             if ctx.fell_back:
183                 logger.warning("device=auto: no CUDA GPU detected (%s) ? FALLING BACK TO
```

```
CPU; WASB "
184                               "inference will be slow. Set indexing.wasb.device=cuda to
require a GPU.", out)
185             return True, out
186
187         def run_predict(self, video_win_path: str, output_win_dir: str,
188                         video_id: Optional[str] = None) -> Optional[str]:
189             """Run WASB natively on a video. Returns the path to the trajectory CSV, or
None.
190
191             Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
WSL
192             runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
the
193             same downstream contract, so the parity comparison is apples-to-apples.
194             """
195             output_dir = os.path.abspath(output_win_dir)
196             os.makedirs(output_dir, exist_ok=True)
197             # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
198             norm = str(video_win_path).replace("\\", "/")
199             stem, _ext = os.path.splitext(os.path.basename(norm))
200             # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
201             key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
202             out_csv = os.path.join(output_dir, f"{key}_wasb.csv")
203
204             if not self._copy_infer_script():
205                 logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
206                 return None
207
208             weights = os.path.expanduser(self.cfg.weights_path)
209             argv = [self.cfg.python_bin, "wasb_infer.py",
210                     "--video", os.path.abspath(str(video_win_path))]
211             frames_dir: Optional[str] = None
212             if self.cfg.stream_video:
213                 # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
214                 argv.append("--stream-video")
215             else:
216                 scratch = self.cfg.scratch_dir or output_dir
217                 frames_dir = os.path.join(scratch, f"{key}_frames")
218                 argv += ["--frames_out_dir", frames_dir]
219             # M4: resolve device="auto" to the effective device (cuda-if-available-else-
cpu); an
220             # explicit cuda/mps/cpu passes through unchanged. Never sends "auto" to
wasb_infer.py.
221             device = self._effective_device()
222             argv += ["--weights", weights,
223                      "--sport", self.cfg.sport,
224                      "--device", device,
225                      "--out", out_csv]
226
227             logger.info("Running native WASB inference (device=%s%s) on %s ...",
228                         device, " [auto?fallback]" if self.cfg.device == AUTO and device !=
"cuda" else "",
229                         os.path.basename(norm))
230             try:
231                 res = self._run(argv, cwd=self._repo_src())
232             except FileNotFoundError:
233                 logger.error("python binary not found: %r (set WASB_PYTHON).",
self.cfg.python_bin)
234                 return None
235             except subprocess.TimeoutExpired:
236                 logger.error("WASB native inference timed out after %ss.",
self.cfg.timeout_sec)
```

```
237                 return None
238             if res.returncode != 0:
239                 logger.error("WASB native inference failed:\n%s", (res.stderr or res.stdout
or "")[-2000:])
240                 return None
241             if not os.path.exists(out_csv):
242                 logger.error("WASB native finished but expected CSV not found at %s",
out_csv)
243                 return None
244             logger.info("WASB trajectory CSV: %s", out_csv)
245 
246             # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
247             # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
248             # (Streaming never materializes frames, so there is nothing to drop on that
path.)
249             if not self.cfg.keep_frames and frames_dir:
250                 logger.info("Cache hygiene: removing frame cache for %s "
251                             "(set indexing.wasb.keep_frames=true to retain).", key)
252                 self._rm_rf(frames_dir)
253             return out_csv
254 
255         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
256         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
257         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
258 
```

13. assistant (2026-06-21T09:05:00.922Z)

Now I have the full picture. Let me verify a few subtleties: the `_probe_cuda` stdout parsing vs healthcheck stdout, the `_drive_success` helper to confirm what the test asserts, and how `healthcheck` relates to `run_predict` in production (whether healthcheck is always called first).

14. user (2026-06-21T09:05:02.344Z)

```
200         """An explicit device=cuda resolves to itself with NO probe subprocess ? unchanged
behaviour
201         (a single _run call: the inference)."""
202         r, _out, _rm, argv, _cwd, _key = _drive_success(
203             NativeWasbConfig(weights_path="/m/w.pth", device="cuda"), tmp_path)
204         assert argv[argv.index("--device") + 1] == "cuda"
205         assert r._cuda_available is None           # never probed for an explicit device
206 
207 
208     # --- run_predict
---------------------------------------------------------------------
209     def _drive_success(cfg, tmp_path, vid="abc123abc123", **patches):
210         """Drive run_predict to success with the subprocess mocked; returns (runner, out,
rm_spy, argv, cwd)."""
211         r = NativeWasbRunner(cfg)
212         key = f"clip__{vid[:12]}"
213         (tmp_path / f"{key}_wasb.csv").write_text("Frame,Visibility,X,Y\n")  # success = CSV
exists
214         with patch.object(r, "_copy_infer_script", return_value=True), \
215              patch.object(r, "_run", return_value=_proc(0)) as run_spy, \
216              patch.object(r, "_rm_rf") as rm_spy:
217             out = r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id=vid)
218         argv = run_spy.call_args[0][0]
219         cwd = run_spy.call_args.kwargs.get("cwd")
220         return r, out, rm_spy, argv, cwd, key
221 
```

```
222
223     def test_run_predict_builds_native_argv_no_wsl(tmp_path):
224         # Pin disk mode here (default is now streaming) so this also covers the
--frames_out_dir path.
225         _, out, rm, argv, cwd, key = _drive_success(
226             NativeWasbConfig(weights_path="/m/w.pth", device="cuda", stream_video=False),
tmp_path)
227         assert out == os.path.join(os.path.abspath(str(tmp_path)), f"{key}_wasb.csv")
228         # No WSL / conda plumbing anywhere in the command.
229         assert "wsl" not in argv
230         assert not any("conda" in a or "/mnt/" in a for a in argv)
231         # It runs the SAME inference core, device-threaded, writing the SAME CSV name as
WSL.
232         assert argv[0] == "python" and argv[1] == "wasb_infer.py"
233         assert "--device" in argv and argv[argv.index("--device") + 1] == "cuda"
234         assert "--weights" in argv and argv[argv.index("--weights") + 1] == "/m/w.pth"
235         assert argv[argv.index("--out") + 1] == out
236         # Disk mode extracts frames + cleans them on success.
237         assert "--frames_out_dir" in argv and "--stream-video" not in argv
238         assert cwd.endswith(os.path.join("src"))
239         rm.assert_called_once()
240
241
242     def test_run_predict_default_is_stream_mode_no_frames_dir(tmp_path):
243         # No stream_video set -> the NATIVE DEFAULT (streaming): no PNG extraction, nothing
to clean.
244         _, out, rm, argv, _cwd, _key =
_drive_success(NativeWasbConfig(weights_path="/m/w.pth"), tmp_path)
245         assert out is not None
246         assert "--stream-video" in argv and "--frames_out_dir" not in argv
247         rm.assert_not_called()  # streaming never materializes a frame cache
248
249
250     def test_run_predict_frames_dir_uses_scratch_and_is_cleaned(tmp_path):
251         """Frames extract under scratch_dir (the box's big NVMe) when set, and the SAME dir
is the
252         one cleaned on success ? a regression that dropped scratch_dir (frames -> small
system disk)
253         or diverged the cleanup target would be a disk-fill leak."""
254         scratch = tmp_path / "scratch"
255         r, out, rm, argv, _cwd, key = _drive_success(
256             NativeWasbConfig(weights_path="/m/w.pth", scratch_dir=str(scratch),
stream_video=False), tmp_path)
257         expected_frames = os.path.join(str(scratch), f"{key}_frames")
258         assert argv[argv.index("--frames_out_dir") + 1] == expected_frames
259         rm.assert_called_once_with(expected_frames)   # cleanup target == extraction target
260
261
262     def test_run_predict_frames_dir_defaults_next_to_output(tmp_path):
263         _, out, rm, argv, _cwd, key = _drive_success(
264             NativeWasbConfig(weights_path="/m/w.pth", stream_video=False), tmp_path)
265         expected_frames = os.path.join(os.path.abspath(str(tmp_path)), f"{key}_frames")
266         assert argv[argv.index("--frames_out_dir") + 1] == expected_frames
267         rm.assert_called_once_with(expected_frames)
268
269
270     def test_copy_infer_script_copies_byte_identical(tmp_path):
271         """Unmocked: the native runner stages the SAME wasb_infer.py the WSL runner runs
(the
272         byte-parity precondition) into <repo>/src/, byte-for-byte."""
273         import filecmp
274         from backend.pipeline.detectors import native_wasb_runner as nwr
275         r = NativeWasbRunner(NativeWasbConfig(repo_dir=str(tmp_path / "repo")))
276         assert r._copy_infer_script() is True
277         dest = tmp_path / "repo" / "src" / "wasb_infer.py"
```

```
278         assert dest.exists() and filecmp.cmp(nwr._INFER_PY, str(dest), shallow=False)
279
280
281     def test_copy_infer_script_false_on_oserror(tmp_path, monkeypatch):
282         r = NativeWasbRunner(NativeWasbConfig(repo_dir=str(tmp_path / "repo")))
283         monkeypatch.setattr("backend.pipeline.detectors.native_wasb_runner.shutil.copy",
284                             lambda *a, **k: (_ for _ in ()).throw(OSError("no space")))
285         assert r._copy_infer_script() is False
286
287
288     def test_run_predict_device_threaded_cpu(tmp_path):
289         _, _out, _rm, argv, _cwd, _key = _drive_success(
```

15. user (2026-06-21T09:05:03.535Z)

```
backend\results.py:6:(e.g. missing API key, WSL healthcheck failure, provider error, etc.).
backend\worker\__main__.py:240:    ok, msg = runner.healthcheck()
backend\reporting\spec.yaml:64:  # provider error / quota / WSL healthcheck issue rather than a
truly empty video.
backend\reporting\spec.yaml:74:      failed WSL healthcheck ? these all return [] silently. Re-
run with -u for
backend\pipeline\segmenters\base.py:30:        "no segments found" from "something went wrong"
(missing key, healthcheck
backend\pipeline\detectors\base.py:11:- Provide a single place to evolve the healthcheck / error
contract and thread
backend\pipeline\detectors\base.py:21:- healthcheck() -> tuple[bool, str]
backend\pipeline\detectors\base.py:31:2. Implement run_predict and healthcheck (return (bool,
str) tuple).
backend\pipeline\detectors\base.py:41:6. Add unit tests that exercise the contract (isinstance,
healthcheck shape, run
backend\pipeline\detectors\base.py:51:- healthcheck can just check "weights file exists and
torch importable + CUDA?".
backend\pipeline\detectors\base.py:57:The healthcheck message is already logged by the hybrids
on failure. The reporting
backend\pipeline\detectors\base.py:179:    def healthcheck(self) -> Tuple[bool, str]:
backend\pipeline\detectors\device.py:9:healthcheck hard-failed on a box with no GPU, so a CPU-
only Linux box (cpu-serving / a
backend\pipeline\detectors\native_wasb_runner.py:101:        # Cached CUDA-availability probe
(used to resolve device="auto"). Set by healthcheck;
backend\pipeline\detectors\native_wasb_runner.py:102:        # lazily probed by run_predict if
healthcheck was skipped. None = not yet probed.
backend\pipeline\detectors\native_wasb_runner.py:123:        """CUDA availability, cached across
healthcheck + run_predict so we probe at most once."""
backend\pipeline\detectors\native_wasb_runner.py:154:    def healthcheck(self) -> Tuple[bool,
str]:
backend\pipeline\detectors\native_wasb_runner.py:171:            return False, "native torch
healthcheck timed out."
backend\pipeline\segmenters\tracknet_hybrid.py:9:The pipeline engine (healthcheck -> trajectory
-> windows -> AI handoff -> DB)
backend\pipeline\segmenters\trajectory_hybrid.py:7:    healthcheck gate -> P2 trajectory ->
candidate windows (+ persistence)
backend\pipeline\segmenters\trajectory_hybrid.py:227:        # --- Phase 1: env healthcheck
(fail fast with a clear message) ---
backend\pipeline\segmenters\trajectory_hybrid.py:228:        ok, msg = runner.healthcheck()
backend\pipeline\segmenters\wasb_hybrid.py:11:The pipeline engine (healthcheck -> trajectory ->
windows -> AI handoff -> DB)
backend\pipeline\detectors\tracknet_runner.py:97:    def healthcheck(self) -> Tuple[bool, str]:
backend\pipeline\detectors\tracknet_runner.py:112:            return False, "WSL healthcheck
timed out."
backend\pipeline\detectors\stub_runner.py:70:    def healthcheck(self) -> Tuple[bool, str]:
backend\pipeline\detectors\wasb_runner.py:72:    def healthcheck(self) -> Tuple[bool, str]:
backend\pipeline\detectors\wasb_runner.py:82:            return False, "WSL healthcheck timed
out."
```

16. assistant (2026-06-21T09:05:06.252Z)

Let me verify the production call path ? `trajectory_hybrid.py` around line 227-228 ? to confirm whether healthcheck is always run before `run_predict` (relevant to cache coherence) and whether a failed healthcheck stops execution.

17. user (2026-06-21T09:05:07.366Z)

```
220                 "detector": self.name,
221                 "velocity_thresh": velocity_thresh,
222                 "merge_gap": merge_gap,
223                 "min_action_window_duration": min_window_duration,
224                 **runner_params,
225             }
226
227             # --- Phase 1: env healthcheck (fail fast with a clear message) ---
228             ok, msg = runner.healthcheck()
229             if not ok:
230                 logger.error("%s detector environment not ready: %s", label, msg)
231                 return []
232             logger.info("%s detector env OK: %s", label, msg)
233
234             # --- Phase 2: trajectory -> candidate rally windows ---
235             # Trajectory detectors process the whole video in one pass (they carry
236             # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
237             t_start = time.time()
238             out_dir = os.path.join(output_base(self.config), self.family)
239             csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
240             if not csv_path:
241                 logger.error("%s produced no trajectory; aborting.", label)
242                 return []
243
244             points = runner.parse_trajectory_csv(csv_path)
245             logger.info("Parsed %d trajectory points (%d visible).",
246                         len(points), sum(1 for p in points if p.visible))
247
248             all_candidate_windows = runner.trajectory_to_action_windows(
249                 points, fps=fps, frame_width=frame_width, chunk_start=0.0,
250                 velocity_thresh=velocity_thresh, merge_gap=merge_gap,
251                 min_window_duration=min_window_duration, chunk_index=0,
252                 max_window_duration=max_window_duration, min_split_gap=window_min_split_gap,
253                 window_hard_cap=window_hard_cap,
254                 window_inactivity_end=window_inactivity_end,
255                 inactivity_rally_thresh=inactivity_rally_thresh,
256                 inactivity_min_density=inactivity_min_density,
257                 window_blob_fallback=window_blob_fallback,
258                 window_blob_factor=window_blob_factor,
259             )
260             logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
261                         label, time.time() - t_start, len(all_candidate_windows))
262
263             if log_candidates:
264                 for cw in all_candidate_windows:
265                     logger.info(f"[{label} rally]
{_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
266                                 f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
267                                 f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")
268
269             # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
270             # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
271             # persistence and the handoff gate below.
272             window_stats = [
273                 self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
```

```
idx_cfg)
274                 for cw in all_candidate_windows
```

18. assistant (2026-06-21T09:05:11.899Z)

This confirms the production path: `healthcheck()` is called at line 228 and `run_predict()` at line 239 on the **same runner instance** ? so in production, the probe cache set in healthcheck is always populated and reused by run_predict. The lazy probe in `run_predict` is only for callers (tests, direct API) that skip healthcheck.

Now let me run the new tests and the full suite to verify green, and check lint/type cleanliness on the changed files.

19. user (2026-06-21T09:05:21.010Z)

```
.........................................                                [100%]
43 passed in 5.15s
```

20. assistant (2026-06-21T09:05:22.708Z)

All 43 tests pass. Let me run the full suite and lint/type checks on the changed files.

21. user (2026-06-21T09:06:46.980Z)

```
======================================================================
 Full test suite:  C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m pytest
-q
 obsreport (optional reporting dep): available
======================================================================
........................................................................ [  6%]
........................................................................ [ 12%]
........................................................................ [ 19%]
........................................................................ [ 25%]
........................................................................ [ 32%]
........................................................................ [ 38%]
........................................................................ [ 44%]
........................................................................ [ 51%]
........................................................................ [ 57%]
........................................................................ [ 64%]
........................................................................ [ 70%]
........................................................................ [ 76%]
........................................................................ [ 83%]
........................................................................ [ 89%]
........................................................................ [ 96%]
.........................................                                [100%]
1125 passed in 80.69s (0:01:20)
======================================================================
 RESULT: PASS - all tests green
======================================================================
```

22. user (2026-06-21T09:06:47.810Z)

```
All checks passed!
---MYPY---
Success: no issues found in 2 source files
```

23. assistant (2026-06-21T09:06:53.496Z)

Full suite green (1125 passed), ruff clean, mypy clean. Now let me carefully verify each of the 5 review points. Let me check one subtle correctness concern: the `_probe_cuda` parses `"cuda True"` from `print('cuda', torch.cuda.is_available())`, while healthcheck parses from `print('torch', ..., 'cuda', ...)`. Let me confirm both produce a substring `"cuda True"` correctly, and check the mps-on-auto and unknown-device edge cases by tracing the logic precisely.

Let me also verify there's no `mps`-availability concern and check how `wasb_infer.py` handles the device argument, to confirm the "auto never reaches it" claim matters.

24. user (2026-06-21T09:06:55.129Z)

```
cuda True
---
torch 1.11.0 cuda False
```

25. user (2026-06-21T09:06:56.450Z)

```
56:def _build_cfg(weights: str, sport: str, device: str = "cuda"):
57:    """Compose the WASB eval config via Hydra, overriding model/weights/device.
59:    ``device`` is ``cuda`` (default ? the historical WSL behaviour, byte-parity preserved),
61:    cpu/mps run with no GPU list so the detector places tensors on ``device``.
64:    gpus = "[0]" if device == "cuda" else "[]"   # single GPU on cuda; none for cpu/mps
70:            f"runner.device={device}",
138:                        frames_in: int, frames_total: int, device: str = "cuda") -> bool:
141:    ``device`` defaults to ``cuda`` so caches written before device-keying (no ``device``
142:    field) stay compatible with a CUDA run ? but a cpu/mps run will NOT reuse a cuda cache
143:    (detections can differ numerically across devices), and vice-versa.
153:        and manifest.get("device", "cuda") == device
376:        device: str = "cuda") -> str:
407:    cfg = _build_cfg(weights, sport, device)
443:        frames_in=frames_in, frames_total=len(frames), device=device,
461:        "device": device,
648:                        device: str = "cuda") -> str:
666:                    frame_loader=frame_loader, source_key=source_key, device=device)
682:    ap.add_argument("--device", default="cuda", choices=["cuda", "mps", "cpu"],
683:                    help="torch device for the detector (default cuda = the WSL parity
path)")
701:                            flush_every=args.flush_every, fresh=args.fresh,
device=args.device)
712:        flush_every=args.flush_every, fresh=args.fresh, device=args.device)
```

26. assistant (2026-06-21T09:07:03.528Z)

This confirms a key point: `wasb_infer.py` line 682-683 declares `--device` with `choices=["cuda", "mps", "cpu"]` ? argparse would **reject** a literal `"auto"` with exit code 2. So the claim "auto must never reach wasb_infer.py" is real and load-bearing: if `auto` leaked through, the inference subprocess would hard-fail. The M4 code correctly resolves `auto` ? cuda/cpu before building argv on both paths.

Now let me trace the edge cases precisely to confirm point 4. Let me verify the `_effective_device` behavior for `mps` and unknown strings, and the torch-import-failure path in healthcheck.

Let me re-examine the healthcheck ordering for the `auto` + torch-import-failure case. At line 173, if `res.returncode != 0` (torch import failed), healthcheck returns `False` early ? before reaching the device-resolution block at line 176. So `_cuda_available` stays `None`. Let me trace what happens next if a caller ignores the healthcheck failure and calls `run_predict` anyway with `device="auto"`.

27. user (2026-06-21T09:07:04.281Z)

```
100
101
102    def test_healthcheck_repo_missing(tmp_path):
103        w = tmp_path / "w.pth.tar"
104        w.write_text("x")
105        cfg = NativeWasbConfig(weights_path=str(w), repo_dir=str(tmp_path / "absent"))
106        ok, msg = NativeWasbRunner(cfg).healthcheck()
107        assert not ok and "repo" in msg.lower()
108
```

```
109
110     def test_healthcheck_torch_import_fails(tmp_path):
111         r = NativeWasbRunner(_ok_cfg(tmp_path))
112         with patch.object(r, "_run", return_value=_proc(1, stderr="ModuleNotFoundError:
torch")):
113             ok, msg = r.healthcheck()
114         assert not ok and "failed" in msg.lower()
115
116
117     def test_healthcheck_cuda_unavailable_when_device_cuda(tmp_path):
118         r = NativeWasbRunner(_ok_cfg(tmp_path, device="cuda"))
119         with patch.object(r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda
False")):
120             ok, msg = r.healthcheck()
121         assert not ok and "cuda" in msg.lower()
122
123
124     def test_healthcheck_cpu_device_ok_without_cuda(tmp_path):
125         r = NativeWasbRunner(_ok_cfg(tmp_path, device="cpu"))
126         with patch.object(r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda
False")):
127             ok, msg = r.healthcheck()
128         assert ok  # cpu run does not require a GPU
129
130
131     def test_healthcheck_python_not_found(tmp_path):
132         r = NativeWasbRunner(_ok_cfg(tmp_path, python_bin="/no/such/python"))
133         with patch.object(r, "_run", side_effect=FileNotFoundError()):
134             ok, msg = r.healthcheck()
135         assert not ok and "python" in msg.lower()
136
137
```

28. assistant (2026-06-21T09:07:15.273Z)

I now have everything I need to complete the review. Let me trace the remaining edge cases precisely by reasoning through the code (no need to run more):

Edge case A ? `auto` + torch import fails (rc!=0) in healthcheck: healthcheck returns `False` at line 173-174 (import failed), `_cuda_available` stays `None`. Production aborts (trajectory_hybrid line 229-231). If a caller skips healthcheck and calls `run_predict` with `device="auto"`: `_effective_device` ? `_cuda_is_available` ? `_probe_cuda` runs its own subprocess (`import torch; print(...)`). That subprocess also fails (rc!=0 since torch unimportable) ? `_probe_cuda` returns `False` ? device resolves to `cpu`. Then the actual inference subprocess will fail too (torch unimportable), and `run_predict` returns `None` gracefully. No crash. Correct.

Edge case B ? `mps`: `_effective_device` line 131: `self.cfg.device == AUTO` is False ? returns `"mps"` verbatim, no probe. healthcheck: `DeviceContext("mps", ...)` ? `cuda_required_but_missing` False, `fell_back` False ? passes, no warning. Correct passthrough.

Edge case C ? unknown device e.g. `"gpu"`: `_effective_device` returns `"gpu"` verbatim (not AUTO). healthcheck: `cuda_required_but_missing` False (not "cuda"), `fell_back` False ? healthcheck passes. Then `wasb_infer.py --device gpu` hits argparse `choices=["cuda","mps","cpu"]` ? subprocess exits rc=2 ? `run_predict` returns `None`. This is a pre-existing behavior (M4 didn't change validation of unknown strings), and it fails loudly at inference, not silently. Acceptable ? noting as a minor.

Edge case D ? warning fires only on real fallback: line 182 `if ctx.fell_back` ? only true when `requested==AUTO and not cuda_available`. cpu/mps/explicit-cuda never set `fell_back`. Correct.

Let me confirm there are no other consumers of `.device` that could leak `auto`, and check the worker `__main__.py` path that also calls healthcheck.

29. user (2026-06-21T09:07:17.275Z)

```
backend/pipeline/detectors/native_wasb_runner.py:15:    strict); threaded into ``wasb_infer.py
--device`` (and so the Hydra ``runner.device`` override).
backend/pipeline/detectors/native_wasb_runner.py:44:from backend.pipeline.detectors.device
import AUTO, DeviceContext
backend/pipeline/detectors/native_wasb_runner.py:128:    def _effective_device(self) -> str:
backend/pipeline/detectors/native_wasb_runner.py:131:        if self.cfg.device == AUTO:
backend/pipeline/detectors/native_wasb_runner.py:133:        return self.cfg.device
backend/pipeline/detectors/native_wasb_runner.py:177:        ctx =
DeviceContext(self.cfg.device, self._cuda_available)
backend/pipeline/detectors/native_wasb_runner.py:180:            # slow-CPU downgrade. Set
indexing.wasb.device=auto for a CPU fallback.
backend/pipeline/detectors/native_wasb_runner.py:184:                                "inference will
be slow. Set indexing.wasb.device=cuda to require a GPU.", out)
backend/pipeline/detectors/native_wasb_runner.py:221:        device = self._effective_device()
backend/pipeline/detectors/native_wasb_runner.py:228:                      device, "
[auto?fallback]" if self.cfg.device == AUTO and device != "cuda" else "",
backend/pipeline/segmenters/fusion_hybrid.py:87:
"fusion_substrate": "wasb", "device": cfg.device}
backend/pipeline/segmenters/wasb_hybrid.py:46:
"detector_impl": "native", "device": cfg.device}
```

30. assistant (2026-06-21T09:07:53.584Z)

Lines 87 (fusion_hybrid) and 46 (wasb_hybrid) put `cfg.device` into `runner_params` for reporting/metadata only ? they don't pass it to argv. The only argv path is `_effective_device()` at line 221. Both `--device` argv construction sites (line 224 via `_effective_device`) are covered. The `auto` string is fully contained.

Let me do one final check ? confirm `_cuda_available` being set to `False` (not `None`) for explicit non-cuda devices in healthcheck doesn't cause an inconsistency, and verify the cache-coherence concern about a `False` cache being reused correctly when device is `auto`.

In healthcheck line 176: `self._cuda_available = "cuda True" in out` is set for **every** device (cuda/cpu/mps/auto) once torch imports successfully. This is a genuine probed value, so the cache is always coherent with reality. For `auto`, run_predict's `_effective_device` ? `_cuda_is_available` sees the cached bool (not None) ? no second probe. Correct and coherent.

One subtle thing to verify: is there any scenario where the cache is set to a **stale** value? The probe reflects the box's GPU at healthcheck time; GPU availability doesn't change within a process lifetime, so no staleness risk. Clean.

The review is complete. All checks pass.

Review: M4 (DeviceContext + WASB CPU-fallback) ? COMPUTE_DECOUPLED_SERVING

Verdict: clean bill. No blockers, no majors. Two nits. The "cuda default unchanged" safety contract holds, `auto` never reaches `wasb_infer.py`, the probe cache is coherent, and the safety/fallback/cache contracts are all pinned by tests that would fail on regression.

Verified locally: full suite **1125 passed** (`python run_all_tests.py`), the M4 tests **43 passed**, `ruff check` clean, `mypy` clean on both changed source files.

1. SAFETY / "cuda default unchanged" ? PASS
- Default is still `cuda`: `backend/config/models.py:72` and `native_wasb_runner.py:67` both default `device="cuda"`; `from_indexing_cfg` falls back to `"cuda"` (`native_wasb_runner.py:88`).
- Explicit `cuda` on a GPU-less box still **hard-fails** the healthcheck: `native_wasb_runner.py:177-181` ? `DeviceContext.cuda_required_but_missing` (`device.py:49-52`) is True only for `requested=="cuda" and not cuda_available`, returning the unchanged `device=cuda but torch.cuda.is_available() is False` error. Pinned by `test_healthcheck_cuda_unavailable_when_device_cuda` and `test_explicit_cuda_is_honored_verbatim`.

- No silent cuda?cpu path: `_effective_device` (`native_wasb_runner.py:128-133`) only consults the probe when `device == "auto"`; explicit `cuda` returns `"cuda"` verbatim. `DeviceContext.effective` (`device.py:39-41`) likewise returns explicit devices verbatim. cpu/mps unchanged (`test_healthcheck_cpu_device_ok_without_cuda`, `test_explicit_cpu_and_mps_pass_through`).

2. "auto" NEVER reaches wasb_infer.py ? PASS (and load-bearing)
Confirmed `wasb_infer.py:682` declares `--device ... choices=["cuda","mps","cpu"]`, so a literal `auto` would make argparse exit rc=2. M4 resolves it before argv on **both** entry orderings:
- run_predict builds `--device` from `device = self._effective_device()` (`native_wasb_runner.py:221-224`), never `self.cfg.device`.
- Healthcheck-first: cache set at `:176`, reused with no 2nd probe. Healthcheck-skipped: `_effective_device`?`_cuda_is_available` lazily probes (`:122-126`). Either way argv gets cuda/cpu. Pinned by `test_run_predict_auto_threads_cpu_when_no_gpu` (`assert "auto" not in argv`) and `test_run_predict_auto_threads_cuda_when_gpu`. The two metadata sinks (`fusion_hybrid.py:87`, `wasb_hybrid.py:46`) only record `cfg.device` for reporting ? they never build argv.

3. PROBE CACHING & CORRECTNESS ? PASS
- Coherent: healthcheck sets `self._cuda_available = "cuda True" in out` (`:176`) for every device once torch imports, from a real probe; `_cuda_is_available` (`:122-126`) reuses it (None?probe-once memoization). GPU availability can't change within a process, so no staleness. Production always runs healthcheck before run_predict on the same instance (`trajectory_hybrid.py:228,239`), so the lazy path is the test/direct-API case only.
- `_probe_cuda` (`:112-120`) treats python-missing/timeout (caught `FileNotFoundError`/`TimeoutExpired` ? `False`), import-error/rc!=0 (`res.returncode == 0 and ...` ? `False`) all as "no cuda" ? auto?cpu, never a crash. Pinned by `test_healthcheck_caches_probe_for_run_predict` (asserts `spy.call_count` stays 1) and `test_run_predict_explicit_cuda_does_not_probe` (`r._cuda_available is None`).

4. EDGE CASES ? PASS
- `auto` + torch import fails (rc!=0) in healthcheck: returns False early at `:173-174`, `_cuda_available` stays None, production aborts. If healthcheck is skipped, the lazy `_probe_cuda` subprocess also fails ? resolves to `cpu`, then the inference subprocess fails and run_predict returns `None` gracefully (`:238-240`). No crash.
- `mps`: passthrough, no probe, no warning (`device.py` mps branch; `test_explicit_cpu_and_mps_pass_through`).
- Warning fires only on real fallback: `:182 if ctx.fell_back` (`device.py:43-46`) is True only for auto-without-cuda. Pinned by `test_healthcheck_auto_falls_back_to_cpu_with_warning`.

5. TEST GAPS / OS ? PASS
All five contracts are pinned by a failing-on-regression test: explicit-cuda-strict, auto?cpu, auto?cuda, no-probe-for-explicit, cache-reuse (enumerated above). `DeviceContext` frozen-ness (`test_is_frozen` ? `FrozenInstanceError`) and all three properties (`effective`/`fell_back`/`cuda_required_but_missing`) covered in `tests/test_device_context.py`. No OS-dependent assertions ? all subprocesses are mocked and Windows paths are normalized (`run_predict :198`); the suite passed here on win32.

Nits (non-blocking, optional)
- **nit** ? Unknown device string (e.g. `"gpu"`): passes healthcheck (not "cuda", not "auto" ? no gate) and only fails later at `wasb_infer.py` argparse (rc=2 ? run_predict returns None). This is pre-existing behavior M4 didn't regress, but since `DeviceContext` is now the device-policy seam, a future hardening could validate `requested ? {auto,cuda,mps,cpu}` in healthcheck for a clearer early error. Not required for M4.
- **nit** ? `_probe_cuda` (`native_wasb_runner.py:115`) and healthcheck (`:165`) use two slightly different print formats, but both yield the `"cuda True"` substring the parsers key on (verified). Harmless duplication; a shared probe-code constant would remove the chance of them drifting apart.

No edits made (review only).